

[← Go Back](#)

Hardware

[Access Barometric Pressure Sensor Data on Nano 33 BLE Sense](#)[Connecting Nano 33 BLE Devices over Bluetooth®](#)[Controlling RGB LED Through Bluetooth®](#)[Nano 33 BLE Sense Rev2 User Manual](#)[Nano 33 BLE Sense Community Projects](#)[Edge Impulse with the Nano 33 BLE Sense](#)[Gesture Recognition with the Nano 33 BLE Sense](#)[Get Started With Machine Learning on Arduino](#)[Getting Started with OpenMV](#)[Reading Temperature & Humidity on Nano 33 BLE Sense Rev2](#)[Connecting Two Nano 33 BLE Sense Boards Through I2C](#)[Accessing Accelerometer Data on Nano 33 BLE Sense Rev2](#)[Accessing Gyroscope Data on Nano 33 BLE Sense Rev2](#)[Accessing Magnetometer Data on Nano 33 BLE Sense Rev2](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)[Controlling the On-Board RGB LED with Microphone](#)[Proximity Detection with](#)[Home](#) / [Hardware](#) / [Nano 33 BLE Sense Rev2](#) / **Getting Started with OpenMV**

ON THIS PAGE

Getting Started with OpenMV

In this tutorial, you will learn how to update the bootloader of your Nano 33 BLE (sense) board so that it may be used with OpenMV IDE, allowing to program it with MicroPython.

Author Benjamin Dannegård, Last revision 12/18/2023
Sebastian Romero

This tutorial will show you how to update the bootloader with an Arduino IDE sketch. This is required to run the OpenMV firmware. It shows you how to connect the board in OpenMV and upload a MicroPython script that controls the built-in RGB LED.

Hardware & Software

Nano 33 BLE or Nano 33 BLE sense

Arduino IDE 1.8.10+ or Arduino Pro IDE 0.0.4+

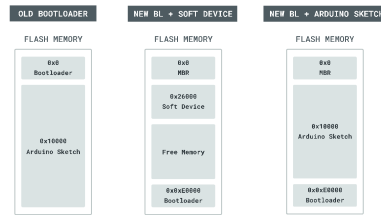
OpenMV IDE 2.6.4+

How the New Bootloader Works

The updated bootloader will first look for a valid sketch at the memory address `0x10000`. This ensures backwards compatibility with the standard sketches uploaded from the Arduino IDE. If no valid sketch is found, because maybe a softdevice has been placed in that portion of

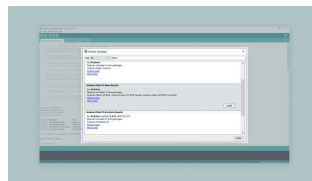
[Hardware & Software](#)[How the New Bootloader Works](#)[Updating the Bootloader](#)[Downloading the OpenMV IDE](#)[Flashing the OpenMV Firmware](#)[Setting up the MicroPython Environment](#)[Creating the Blink Script](#)[Uploading the Script in OpenMV](#) +[Conclusion](#)[Help](#)

valid firmware at `0x26000` and this is the one that you will upload as part of this tutorial.



Updating the Bootloader

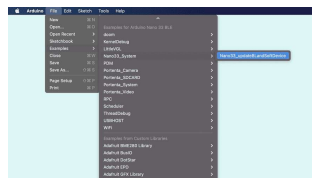
With your board connected, open up the Arduino IDE. This step is the same for both the IDE 1 and the IDE 2. Open the board manager and search for "**mbed**". Find the "Arduino Mbed OS Nano Boards" Board Package and click on "**Install**" to install the latest version.



Arduino IDE board manager

Then, open:

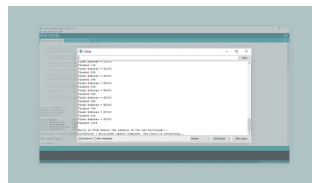
File>Examples>Nano33_System>Nano33_updateBLandSoftDevice



Arduino IDE examples, highlighting the Nano33_updateBootloader sketch

Upload the sketch to your board and

bootloader yet, in the serial monitor you will get the option to do so. Enter "y" into the serial monitor and press the return key. Once the process is done, you will be asked if you want to install the softdevice. Enter "y" and press the return key. Wait for the update to finish. Your board will restart when the process is done and put itself into bootloader mode.



Arduino IDE serial monitor
after the bootloader has
been updated

Downloading the OpenMV IDE

Before you can start programming OpenMV scripts for the Nano 33 BLE sense you need to download and install the OpenMV IDE.

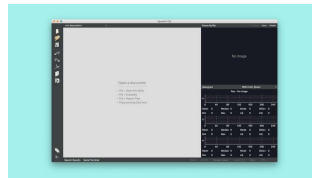
Open the [OpenMV download](#) page in your browser, download the version that you need for your operating system and follow the instructions of the installer.

Flashing the OpenMV Firmware

Connect the Nano 33 BLE sense to your computer via the USB cable if you haven't done so yet. Make sure you have installed the latest bootloader and softdevice as indicated in the previous step.

After installing the bootloader, you

mode if necessary. The board automatically goes into bootloader mode after the update is finished. However, if you restarted the board in the mean time, double click the reset button to bring it into bootloader mode.



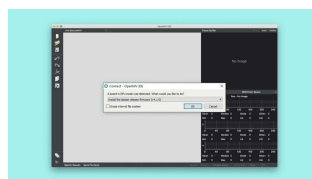
The OpenMV IDE after starting it

Click on the "connect" symbol at the bottom of the left toolbar.



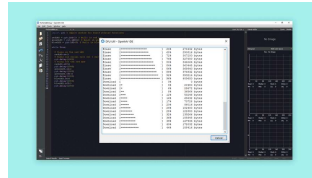
Click the connect button to attach the Nano 33 BLE Sense to the OpenMV IDE

A pop-up will ask you how you would like to proceed "DFU bootloader(s) found. What would you like to do?". Select "Reset Firmware to Release Version". This will install the latest OpenMV firmware on the Nano. If it asks you whether it should erase the internal file system you can click "No".



Install the latest version of the OpenMV firmware

A terminal window will open which shows you the upload progress. You will see a message saying "DFU firmware update complete!" when the process is done.



Installing firmware on Nano 33 board in OpenMV

Installing the OpenMV firmware will overwrite any existing sketches in the internal flash of your board. Also the board's port won't be accessible in the Arduino IDE anymore. To re-flash the board with an Arduino firmware you need to put the board into bootloader mode. To do so double press the reset button on the board. The built-in yellow LED will start fading in and out.

After confirming the completion dialog the board should already be connected to the OpenMV IDE, otherwise click the "connect" button once again.



When the board is successfully connected to the OpenMV IDE a green play button appears in the lower left

Setting up the MicroPython Environment

Now let's create a version of the classic Arduino blink example with MicroPython in OpenMV. First, we need to import some modules in our script, to help us control the board with MicroPython. We do this by adding these lines of code at the beginning of our script.

```
1 import time
2 from board import LED
```

Importing `LED` from the board module, allows us to control the built-in LED on the board. The `time` module will help us with timed functions, such as a delay.

Creating the Blink Script

Now let's take a look at how to create the main loop of the script. The main loop will turn on the LED in a specific color, wait a short moment, turn it off and move on to a different color. But first, we need to define the variables that control the LED.

```
1 led_red = LED(1)
2 led_green = LED(2)
3 led_blue = LED(3)
4 led_yellow = LED(4)
```

The built-in LEDs on the board can now be easily controlled with these different variables. Each variable corresponds to one color of the built-

```
1 while (True):
2     # Turns on the blue LED
3     led_blue.on()
4     # Adds a 250 millisecond delay
5     time.sleep_ms(250)
6     # Turns off the blue LED
7     led_blue.off()
8
9     led_red.on()
10    time.sleep_ms(250)
11    led_red.off()
12
13    led_green.on()
14    time.sleep_ms(250)
15    led_green.off()
16
17    led_yellow.on()
18    time.sleep_ms(250)
19    led_yellow.off()
20
21    time.sleep_ms(500)
```

We use the function

`time.sleep_ms()` to determine the length of the delay between turning the LED on and off. The number inside the parentheses indicates how many milliseconds the delay should last. Putting the code inside a `while (true)` function will make it run in a continuous loop.

Uploading the Script in OpenMV

To upload the script with OpenMV, enter all the code previously explained into an empty script, then simply press the green play button in the lower left corner.



OpenMV highlight green
play button in lower left
corner

The complete script is as follows:

```
1 import time
2 from board import LED
3
4 led_red = LED(1)
5 led_green = LED(2)
6 led_blue = LED(3)
7 led_yellow = LED(4)
8
9 while (True):
10     # Turns on the blue
11     led_blue.on()
12     # Adds a 250 millis
13     time.sleep_ms(250)
14     # Turns off the blue
15     led_blue.off()
16
17     led_red.on()
18     time.sleep_ms(250)
19     led_red.off()
20
21     led_green.on()
22     time.sleep_ms(250)
23     led_green.off()
24
25     led_yellow.on()
26     time.sleep_ms(250)
27     led_yellow.off()
28
```

If everything went correctly the built-in LEDs should blink in the color sequence indicated in the code.

Troubleshoot

Here are some reasons why the softdevice may have to be re-flashed:

- You would like to use a custom softdevice

- You uploaded a custom sketch from the Arduino IDE. Sketches

SoftDevice. If you want to be able to use OpenMV again, you will need to re-flash the Soft Device.

If the code is not working, there are some common issues you can troubleshoot:

You haven't installed the Board Package required for the board so the updater sketch doesn't show up.

You haven't opened the Serial Monitor to initialize the updater sketch.

You haven't uploaded the latest OpenMV firmware for the board.

Conclusion

In this tutorial you learned how to update the bootloader and install a softdevice. You were also introduced to OpenMV and MicroPython scripts. Now that you know how to use these, you can explore more of MicroPython's capabilities, check their documentation [here](#).

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

